PROJECT REPORT

Reading Between the Texts

Probabilistic SMS Spam Detection

Telling genuine messages from spam with prior probability, Bayes' theorem and Multinomial Naive Bayes

Submitted in partial fulfilment of the requirements for

BAMAT207 - Probability and Statistics

Student name	Kavin Raj SK
Registration number	25BCE2866
Programme / Department	B.TECH, CSE--CORE
Dataset	UCI SMS Spam Collection (ID 228) - 5,574 labelled messages
Date of submission	27 October 2026
Companion files	Executed notebook - Python script - Figures - Outputs - Trained model - GitHub README

All results in this report were generated by running the accompanying code with random_state = 2866.

Declaration and Reproducibility Note

I declare that this project report, the accompanying notebook and the Python code constitute my own work for BAMAT207 - Probability and Statistics. The dataset is the public UCI SMS Spam Collection; all other sources consulted are listed in the References. Every computed value reported here was produced by executing the submitted code on that dataset with the cleaning policy and the random seed (2866) stated in Chapter 4, and the verified outputs are collected in Appendix B for checking.

Kavin Raj SK (25BCE2866) | B.TECH, CSE--CORE | 27 October 2026

Abstract

Almost everyone has received a text message they never asked for - a prize they never entered for, an 'urgent' warning, a link best left unclicked. This project asks a deliberately simple question: how far can plain probability go in separating such spam from genuine (ham) messages? Using the UCI SMS Spam Collection, I first reduced the 5,574 labelled messages to 5,160 unique texts (4,518 ham and 642 spam) by trimming whitespace and removing duplicates, and only then started counting.

The statistics come before the machine learning. From the class counts I obtained the prior probabilities P(Ham) = 0.8755814 and P(Spam) = 0.1244186. For the five suspicious words named in the activity sheet - free, prize, win, urgent and offer - I counted how many messages contain the whole word and computed P(Spam | Word) and P(Ham | Word), adding a 'lift' ratio that shows how much each word raises the odds of spam over the prior. For 'free' I rebuilt the same answer with Bayes' theorem and matched the direct count exactly (P(Spam | Free) = 0.7236181). Treating the label as a Bernoulli random variable Y (1 = spam, 0 = ham) gave E(Y) = 0.1244186 and Var(Y) = 0.1089386.

The classifier then applies the same Bayesian idea to every word at once. A stratified 80:20 split (random_state = 2866, taken from my registration number) produced 4,128 training and 1,032 test messages. CountVectorizer, fitted on the training text only, built a vocabulary of 7,597 tokens, and a Multinomial Naive Bayes model was trained on the resulting word counts. On the unseen test set it reached 98.84% accuracy, with spam precision 98.33%, recall 92.19% and F1-score 95.16%; the confusion matrix is [[902, 2], [10, 118]]. The activity sheet's sample message - 'Congratulations! You have won a free prize. Claim now.' - is classified as spam with P(Spam | Message) = 0.999999641. Four extra messages of my own show the system behaving sensibly in both directions, and an inspection of the 12 mistakes shows where a word-count model reaches its limits.

Keywords: SMS spam; ham; prior probability; conditional probability; Bayes' theorem; Bernoulli random variable; expectation; variance; CountVectorizer; Multinomial Naive Bayes; confusion matrix.

Table of Contents

Declaration and Reproducibility Note . . . 2

Abstract . . . 2

Table of Contents . . . 3

Chapter 1 - Introduction . . . 6

1.1 Why spam filtering is really a probability problem . . . 6

1.2 Problem statement . . . 6

1.3 Aim and objectives . . . 6

1.4 Scope and assumptions . . . 6

1.5 Where an approach like this is useful . . . 7

1.6 A note on reproducibility - and honesty about the numbers . . . 7

Chapter 2 - Dataset Description and Preparation . . . 8

2.1 Source and structure . . . 8

2.2 Loading the data - two routes, one result . . . 8

2.3 First inspection: is anything missing? . . . 8

2.4 Removing duplicates . . . 8

2.5 Label encoding . . . 8

2.6 Cleaned dataset summary . . . 9

2.7 An extra look at the data: how long is a spam message? . . . 9

Chapter 3 - Probability and Statistical Analysis . . . 11

3.1 Notation and the counting rule . . . 11

3.2 Prior probabilities: the starting belief . . . 11

3.3 Conditional probability: what changes when a word appears? . . . 11

3.4 Bayes' theorem: reaching P(Spam | Free) by a second road 12

3.5 The label as a random variable: expectation and variance 13

3.6 A contrast I added: three everyday words 13

3.7 How to read these probabilities - carefully 14

Chapter 4 - Methodology: From Counts to a Classifier 15

4.1 The pipeline at a glance 15

4.2 Splitting the data: stratified 80:20 with my own seed 15

4.3 Turning text into numbers: CountVectorizer 15

4.4 Multinomial Naive Bayes: Bayes' theorem with thousands of words 15

4.5 Algorithm 16

4.6 Safeguards: leakage and reproducibility 16

Chapter 5 - Python Implementation 17

5.1 Environment and libraries 17

5.2 How the program is organised 17

5.3 Running it 17

5.4 Inputs and outputs 17

Chapter 6 - Model Evaluation and Results 18

6.1 Training and test partitions 18

6.2 Metrics, with the arithmetic shown 18

6.3 Confusion matrix 18

6.4 Classification report 19

6.5 Error analysis: reading the 12 mistakes 19

Chapter 7 - Real-Time SMS Testing 20

7.1 The message specified in the activity sheet . . . 20

7.2 Four more messages of my own design . . . 20

7.3 Testing a message yourself . . . 21

Chapter 8 - Results and Discussion . . . 21

8.1 The headline results in one place . . . 21

8.2 How the statistics and the classifier fit together . . . 22

8.3 Strengths of the approach . . . 22

8.4 Limitations and validity considerations . . . 22

Chapter 9 - Conclusion and Future Scope . . . 22

9.1 Conclusion . . . 22

9.2 Future scope . . . 23

9.3 What this project taught me . . . 23

References . . . 24

Appendix A - Complete Python Code (sms_spam_detection.py) . . . 25

Appendix B - Verified Output Snapshot . . . 31

Appendix C - Deliverables and Submission Checklist . . . 32

Figures: 1 Class distribution - 2 Message length by class - 3 P(Spam | Word) - 4 System workflow - 5 Confusion matrix. Tables 1-19 are numbered in their captions.

Chapter 1 - Introduction

1.1 Why spam filtering is really a probability problem

A text message gives a filter very little to work with: a handful of words, no formatting, often no punctuation worth trusting. Yet a person glancing at 'WINNER! Claim your free prize now' and at 'See you at the station at six' has no trouble telling which one is unwanted. The difference is not magic - it is experience. Some words turn up in spam far more often than in ordinary conversation, and we have quietly learned those frequencies.

Probability lets us write that experience down as numbers. Before reading a message we have a starting belief about how common spam is: the prior. Reading the words gives evidence. Bayes' theorem is the rule for combining the starting belief with the evidence to get an updated belief, and Naive Bayes is what happens when we apply that rule to every word in the message at once. This project follows exactly that path, from counting single words by hand to training a classifier that counts thousands of them.

1.2 Problem statement

Design and test a Python system that receives an SMS message and labels it Spam (unwanted) or Ham (genuine). The system must be built on the labelled UCI SMS Spam Collection, must expose the probability calculations behind it - priors, word-level conditional probabilities, a Bayes-theorem verification, and the expectation and variance of the spam indicator - and must report, for any new message, both the predicted label and the probability attached to each class.

1.3 Aim and objectives

The aim is to show, end to end and with reproducible numbers, how the probability and statistics of BAMAT207 support a working text classifier. In practical terms I set out to:

- Obtain the SMS Spam Collection from UCI (dataset ID 228), trying ucimlrepo first and falling back to the official ZIP so the work can be repeated offline.
- Inspect the raw file, confirm the missing-value counts, normalise labels and text, remove duplicate messages, and encode Spam = 1 and Ham = 0 (Activity 1).
- Compute the ham/spam counts, percentages and prior probabilities, and chart the class distribution (Activity 2).
- For free, prize, win, urgent and offer, compute P(Spam | Word) and P(Ham | Word) from message counts, and verify P(Spam | Free) using Bayes' theorem (Activity 3).
- Define the random variable Y that marks spam, and obtain E(Y) and Var(Y) both from the Bernoulli formulas and from NumPy (Activity 4).
- Split the cleaned data 80:20 with stratified sampling and a fixed seed, and convert text to word counts with CountVectorizer (Activity 5).
- Train a Multinomial Naive Bayes classifier on the training counts (Activity 6).
- Evaluate it with accuracy, precision, recall, F1-score and a confusion matrix, and read actual examples of both kinds of error (Activity 7).
- Score new messages in real time - the activity sheet's example plus four of my own - printing P(Spam | Message) and P(Ham | Message) (Activity 8).
- Package the notebook, script, figures, outputs and trained model so the project can be uploaded to GitHub and re-run by anyone.

1.4 Scope and assumptions

- Binary classification of English SMS text only: no links are followed, no sender or timing metadata is available in this dataset, and no personal data is collected.
- Word-level probabilities use a whole-word, case-insensitive rule: FREE and free match, freedom does not, and repeated occurrences inside one message count once.
- One fixed stratified split is used for the headline metrics; the seed (2866) and cleaning policy are recorded so the split can be reproduced exactly.
- The model is a teaching baseline. It is not presented as a replacement for a production spam filter or a modern deep-learning system, and its predicted probabilities are model estimates, not calibrated real-world risk.

1.5 Where an approach like this is useful

- Pre-filtering incoming SMS so obvious promotions and scams never reach the inbox.
- Flagging borderline messages for human review when the spam probability is high but not decisive.
- As a transparent teaching example: every quantity in the pipeline can be traced back to a count.
- As a baseline against which phishing-SMS, email-spam or notification-screening systems can be compared.

1.6 A note on reproducibility - and honesty about the numbers

Every number in Chapters 2, 3, 6 and 7 was produced by running the accompanying code (sms_spam_detection.py and the notebook) on the downloaded UCI file with the settings stated in Chapter 4. Where the activity sheet prints an example output - for instance P(Spam | Message) = 0.96 - I treat that as a format illustration only and report the value my fitted model actually produced. Appendix B collects the verified outputs in one place so a reader can check any figure in seconds.

Chapter 2 - Dataset Description and Preparation

2.1 Source and structure

The data is the SMS Spam Collection maintained by the UCI Machine Learning Repository (dataset ID 228), a public corpus of 5,574 SMS messages each labelled ham or spam [1][2]. The downloadable ZIP contains a tab-separated file named SMSSpamCollection plus a readme. There is no header row: the first field on each line is the label and the second is the raw message, so the loader names the two columns label and message explicitly.

Table 1. Dataset at a glance.

Attribute	Details
Source	UCI Machine Learning Repository - SMS Spam Collection (ID 228)
Raw size	5,574 labelled SMS messages (4,827 ham, 747 spam)
Fields per record	label (ham / spam) and message (free text)
Task	Binary classification: Spam = 1, Ham = 0
Model input	Sparse word-count matrix from CountVectorizer
Cleaned size used in this report	5,160 unique messages (4,518 ham, 642 spam)

2.2 Loading the data - two routes, one result

The program first calls fetch_ucirepo(id=228) from the ucimlrepo package [3]. Because an API can rename its columns or be unreachable in an offline lab, the loader checks the returned frame for a column whose values are exactly {ham, spam} and, if the online route fails for any reason, reads the SMSSpamCollection member of the official ZIP stored in the project's data folder instead. In the run recorded for this report the API route was unavailable, so the ZIP fallback was used; both routes deliver the same 5,574 rows, and Appendix B records which one produced the numbers. Either way, the data that reaches the cleaning step has the same shape: 5,574 rows and 2 columns.

2.3 First inspection: is anything missing?

Before changing anything I looked at the raw frame. pandas reports zero missing values in both the label and the message columns, so no row had to be discarded for incompleteness. Labels were then lower-cased and stripped, and message text was trimmed of leading and trailing whitespace. Guards for empty messages and for labels outside {ham, spam} are present in the code, but in this file they removed nothing - a useful thing to state explicitly, because it means every difference between the raw and cleaned row counts is explained by the next step alone.

2.4 Removing duplicates

The activity sheet asks for duplicate messages to be removed, and there are two good reasons to obey literally. First, a repeated text would be counted several times in the probability tables, inflating the influence of whatever campaign sent it. Second, if duplicates survived until the train-test split, the identical text could appear on both sides and quietly flatter the test score. After whitespace trimming, 414 messages were exact repeats of an earlier text and were dropped (before trimming the count is 403; the extra 11 differed only by stray edge spaces, which normalisation erases). I also checked for the awkward case of the same normalised text carrying both labels: it never happens in this corpus, so keeping the first copy with drop_duplicates(subset=["message"], keep="first") throws away repetition, not information.

2.5 Label encoding

Spam = 1 Ham = 0

The mapping {"ham": 0, "spam": 1} is stored in a new column, label_encoded, while the original text label is kept for readable tables. The encoding does double duty: scikit-learn needs numeric targets, and in

Section 3.5 the same 0/1 column becomes the Bernoulli random variable Y.

2.6 Cleaned dataset summary

Table 2. From raw file to analysis-ready data.

Check / quantity	Result	What it tells us
Raw messages	5,574	Matches the UCI listing and the activity sheet.
Missing labels / messages	0 / 0	Nothing removed for incompleteness.
Duplicate messages removed	414	Repeated texts after whitespace trimming.
Cleaned messages (N)	5,160	The population for every calculation that follows.
Ham (0)	4,518 (87.56%)	Genuine messages dominate the corpus.
Spam (1)	642 (12.44%)	The minority class - hence priors well away from 50:50.
Duplicates remaining	0	Each cleaned text is unique.

One consequence deserves emphasis: de-duplication removes slightly more spam than ham in proportion (the raw file is 13.40% spam; the cleaned file is 12.44%), because bulk campaigns repeat themselves. All probabilities in this report are therefore computed on the cleaned 5,160 messages, and Figure 1 shows the imbalance that the rest of the project has to live with.

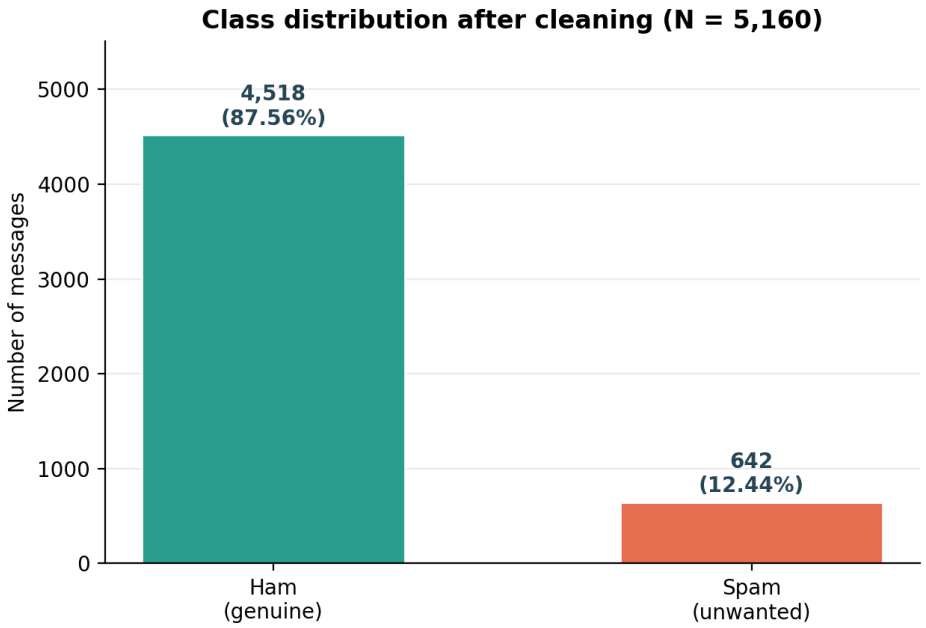


Figure 1. Class distribution of the cleaned dataset (N = 5,160). Ham outnumbers spam by roughly seven to one.

2.7 An extra look at the data: how long is a spam message?

Length is not a feature the classifier uses, but a quick descriptive check is good statistical hygiene - and it immediately shows the classes behaving differently. Genuine texts are short and conversational: a median of 53 characters (11 words). Spam has a sales pitch to fit in - the hook, the prize and the instruction to call or text back - and its median stretches to 148 characters (25 words), close to the classic 160-character SMS limit that much of this historical corpus was written under. Figure 2 shows the two distributions: ham piled up at short lengths, spam spread towards the right. The longest ham message (910 characters) is an outlier chain message; medians describe these skewed distributions better than means, so both are reported.

Table 3. Message-length statistics by class (cleaned data; my own extension).

Class	Mean chars	Median chars	Min-Max chars	Mean words	Median words
Ham	70.9	53	2-910	14.2	11
Spam	137.4	148	13-223	23.7	25

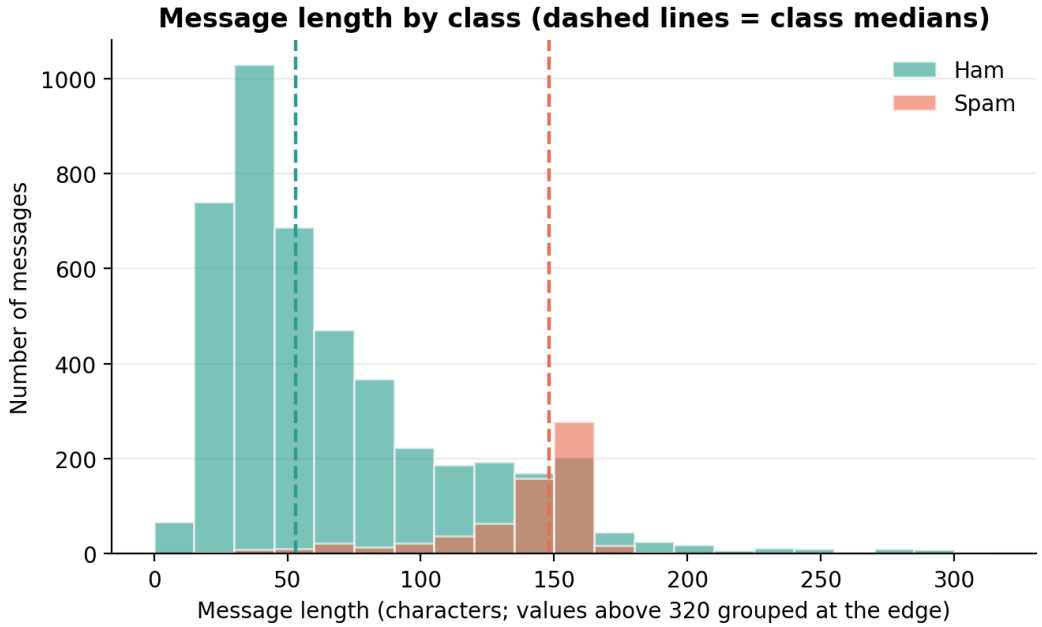


Figure 2. Message length by class. Dashed lines mark the class medians (ham 53, spam 148 characters).

Chapter 3 - Probability and Statistical Analysis

3.1 Notation and the counting rule

Throughout this chapter N = 5,160 is the number of cleaned messages, N_ham = 4,518 and N_spam = 642. For a suspicious word W, a message is counted as 'containing W' when the complete word occurs at least once, with case ignored and word boundaries enforced by a regular expression. This is a message-level count: 'free free FREE' in one text contributes one, not three. The rule matters - a substring rule would also catch freedom, freeloader and PICSFREE1-style tokens, and would produce different (harder to interpret) probabilities.

3.2 Prior probabilities: the starting belief

The empirical probability of an event in a finite dataset is its count divided by the total. Applied to the classes themselves, before any message is read, it gives the priors:

P(Spam) = N_spam / N = 642 / 5,160 = 0.1244186

P(Ham) = N_ham / N = 4,518 / 5,160 = 0.8755814

In percentages, 12.44% of cleaned messages are spam and 87.56% are ham, and the two sum to 1 as they must. The priors also set a sobering baseline for Chapter 6: a 'classifier' that always answers ham would already be right 87.56% of the time while catching no spam at all. That is why accuracy alone is never reported without precision and recall for the spam class.

3.3 Conditional probability: what changes when a word appears?

Conditional probability measures an event after some evidence is known:

P(A | B) = P(A and B) / P(B), provided P(B) > 0

With message-level counting, P(Spam | Word) is simply the fraction of messages containing the word that are labelled spam, and P(Ham | Word) is its complement. Table 4 gives the full picture for the five words named in the activity sheet. The final column is my addition: lift divides P(Spam | Word) by the prior P(Spam), so a lift of 6 means 'seeing this word makes spam about six times as likely as it was before we looked'.

Table 4. Word-level conditional probabilities for the five suspicious words (cleaned data, whole-word matching).

Word	Msgs with word	Spam	Ham	P(Spam | Word)	P(Ham | Word)	Lift vs prior
free	199	144	55	0.7236 (72.36%)	0.2764 (27.64%)	x5.82
prize	70	70	0	1.0000 (100.00%)	0.0000 (0.00%)	x8.04
win	53	47	6	0.8868 (88.68%)	0.1132 (11.32%)	x7.13
urgent	61	56	5	0.9180 (91.80%)	0.0820 (8.20%)	x7.38
offer	30	24	6	0.8000 (80.00%)	0.2000 (20.00%)	x6.43

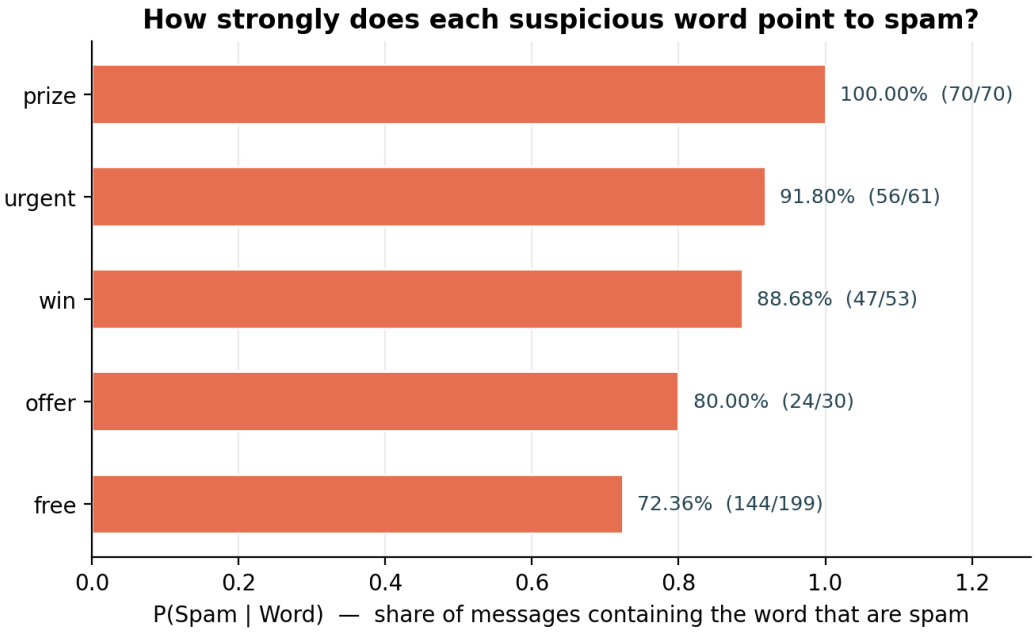


Figure 3. P(Spam | Word) for the five suspicious words, with the raw counts behind each bar.

Reading the table word by word:

- free - the weakest of the five, in relative terms: 144 of 199 matching messages are spam (72.36%), but a substantial 55 are genuine ('free' to call, a free evening, free of cost). Evidence, not a verdict.
- prize - the strongest signal in this corpus: all 70 messages containing prize are spam (100.00%). That is a statement about this sample, not a law of language - the next genuine competition message would break it.
- win - 47 of 53 are spam (88.68%); the 6 ham cases are mostly people talking about sport and games, which is exactly the ambiguity a one-word rule cannot resolve.
- urgent - 56 of 61 are spam (91.80%). Manufactured urgency is a persuasion device, and the counts show it.
- offer - 24 of 30 are spam (80.00%), on the smallest base of the five, so this estimate carries the widest sampling uncertainty.

Two lessons carry forward. First, no single word except prize (on this data) separates the classes cleanly; a classifier must combine many weak-to-strong signals. Second, the denominators are modest - 30 messages for offer - so these are estimates from one corpus, quoted with their counts rather than as naked percentages.

3.4 Bayes' theorem: reaching P(Spam | Free) by a second road

Bayes' theorem reverses a conditional probability. Instead of counting spam among free-messages directly, we assemble the answer from how often free appears inside spam, how common spam is to begin with, and how common free is overall:

P(Spam | Free) = [P(Free | Spam) x P(Spam)] / P(Free)

Table 5. Ingredients of the Bayes calculation for the word free.

Quantity	Count-based calculation	Value
P(Free | Spam) (likelihood)	144 spam messages with free / 642 spam messages	0.2242991
P(Spam) (prior)	642 / 5,160	0.1244186
P(Free) (marginal)	199 messages with free / 5,160	0.0385659
P(Free and Spam) (joint)	144 / 5,160	0.0279070

1. Likelihood: P(Free | Spam) = 144 / 642 = 0.2242991 - about 22.4% of spam contains the word free.
2. Weight by the prior: 0.2242991 x 0.1244186 = 0.0279070, the joint probability of a message being both spam and containing free.
3. Normalise by the marginal: P(Spam | Free) = 0.0279070 / 0.0385659 = 0.7236181.

Verification: the direct count gives 144 / 199 = 0.7236181 - identical to the Bayes result to every digit shown, as it must be, since Bayes' theorem is an algebraic rearrangement of the same counts. The complement is P(Ham | Free) = 55 / 199 = 0.2763819 (27.64%). The program asserts the equality numerically (difference below 1e-9), so a future edit that breaks the arithmetic will fail loudly instead of silently mis-reporting.

3.5 The label as a random variable: expectation and variance

Draw one cleaned message at random and record Y = 1 if it is spam and Y = 0 if it is ham. Y follows a Bernoulli distribution with success probability p = P(Spam):

Table 6. Probability mass function of Y.

y	Meaning	P(Y = y)
0	Ham (genuine)	1 - p = 0.8755814
1	Spam (unwanted)	p = 0.1244186

3.5.1 Expectation

$$E(Y) = (0 \times P(Y=0)) + (1 \times P(Y=1)) = p = 0.1244186$$

NumPy confirms it from the encoded column: mean(Y) = 0.1244186. The expectation of an indicator is just the probability of the event it indicates - here, the long-run fraction of spam. In everyday units, about one message in eight in this corpus is spam.

3.5.2 Variance

For a Bernoulli variable, E(Y^2) = p (since Y^2 = Y), so:

$$Var(Y) = E(Y^2) - [E(Y)]^2 = p - p^2 = p(1-p) = 0.1244186 \times 0.8755814 = 0.1089386$$

NumPy's population variance (ddof=0, treating the 5,160 messages as the whole population of interest) gives the same 0.1089386, and SD(Y) = 0.3300585. The variance is well below the Bernoulli maximum of 0.25 (reached at p = 0.5): because spam is relatively rare, Y is usually 0 and consequently less variable than a fair coin.

3.6 A contrast I added: three everyday words

Suspicious words were chosen for the activity because they are suspicious. To keep myself honest I ran the identical calculation for three ordinary words. claim turns out to be as one-sided as prize in this corpus (91 of 91 messages are spam), but call (61.01% spam) and text (58.60% spam) appear constantly in genuine conversation too. A word's value as evidence depends on both how spammy it is and how often it occurs at all - which is precisely the trade-off the classifier in Chapter 4 weighs automatically.

Table 7. The same analysis for three bonus words (extension beyond the activity sheet).

Word	Msgs with word	Spam	Ham	P(Spam | Word)	Lift vs prior
claim	91	91	0	1.0000 (100.00%)	x8.04
call	454	277	177	0.6101 (61.01%)	x4.90
text	157	92	65	0.5860 (58.60%)	x4.71

3.7 How to read these probabilities - carefully

- They describe this cleaned sample under one stated counting rule. A different corpus, year or matching rule gives different values.
- P(Spam | Prize) = 1.0000 means 'no ham example in 5,160 messages', not 'impossible'. With 70 observations, absence of evidence is not certainty.
- The priors inherit the corpus design: UCI collected more ham than a heavy spam target might receive, so P(Spam) = 0.1244 is a property of the data, not of the reader's phone.
- A single word ignores context, word order and the rest of the message. Section 3.3 is the intuition; the classifier is the generalisation.

Chapter 4 - Methodology: From Counts to a Classifier

4.1 The pipeline at a glance

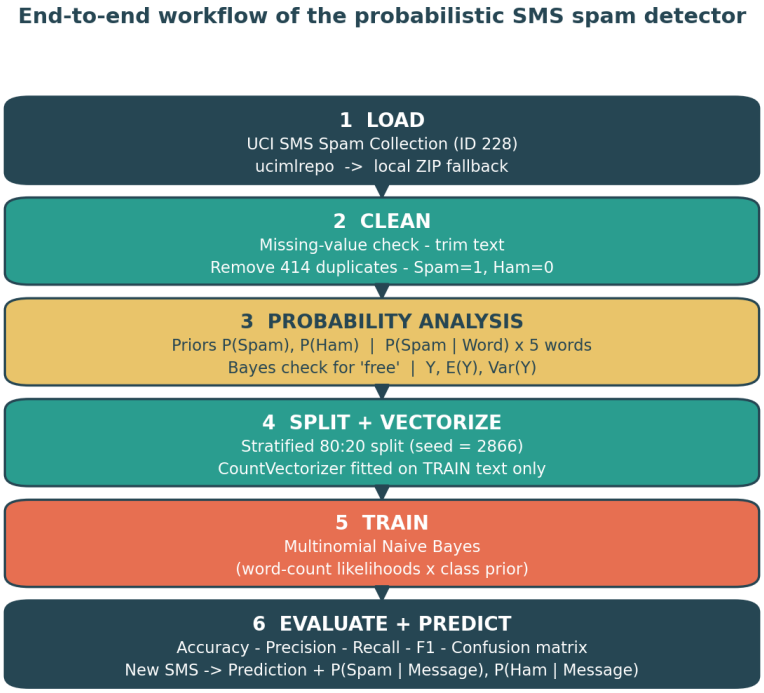


Figure 4. End-to-end workflow. Stage 3 is the hand-computed statistics of Chapter 3; stages 4-6 build and judge the classifier.

4.2 Splitting the data: stratified 80:20 with my own seed

The cleaned messages were divided with train_test_split(test_size=0.20, random_state=2866, stratify=y). Stratification preserves the 87.56 : 12.44 class ratio inside both parts, so the test set contains a fair share of spam (128 messages) instead of leaving the count to chance. The seed 2866 - the last four digits of my registration number 25BCE2866 - is an arbitrary but fixed choice: any reader who sets the same seed on the same cleaned data gets my exact split, and my split differs from a classmate's default-seed split, so our test metrics are genuinely our own. The result is 4,128 training messages (3,614 ham, 514 spam) and 1,032 test messages (904 ham, 128 spam).

4.3 Turning text into numbers: CountVectorizer

Classifiers consume numbers, not sentences. CountVectorizer (lowercase=True) builds a vocabulary of tokens from the training messages - 7,597 of them in this run - and represents each message as a sparse vector counting how often each token occurs [4]. Two discipline points matter more than the mechanics: the vectorizer is fitted on training text only (fit_transform), and the test set - and later, any new SMS - is only transformed. Fitting on all data would leak test vocabulary into the model and inflate the evaluation; re-fitting per new message would change the feature space underneath a trained model and make its probabilities meaningless.

4.4 Multinomial Naive Bayes: Bayes' theorem with thousands of words

For a class c (ham or spam) and a message represented by token counts x = (x_1, ..., x_V), Multinomial Naive Bayes scores each class by [5]:

P(c | x) is proportional to P(c) x product over all tokens j of P(token_j | c) raised to x_j

and predicts the class with the larger (normalised) score. Compare this with Section 3.4: the same prior P(c) appears, and the single likelihood P(Free | Spam) is replaced by a product of likelihoods - one per token. The 'naive' assumption is that, given the class, tokens occur independently; nobody believes words

are truly independent ('free' and 'prize' clearly travel together), but the simplification makes the model fast, stable on small data and, in practice, hard to beat as a text baseline. Default Laplace smoothing (alpha = 1.0) prevents a single unseen-in-class word from zeroing a class score.

4.5 Algorithm

1. Load the UCI data (ucimlrepo, else local ZIP) into label/message columns.
2. Normalise labels and text, drop unusable rows, remove duplicate messages, encode Spam = 1, Ham = 0.
3. Compute priors, the class-distribution chart and message-length summary.
4. For each suspicious word, count matching spam/ham messages; compute P(Word), P(Word | Spam), P(Word | Ham), P(Spam | Word), P(Ham | Word) and lift; verify 'free' with Bayes' theorem.
5. Compute E(Y) and Var(Y) for the Bernoulli spam indicator; cross-check with NumPy.
6. Stratified 80:20 split (seed 2866); fit CountVectorizer on training text; transform both parts.
7. Fit MultinomialNB on the training matrix; predict test labels and class probabilities.
8. Compute accuracy, spam precision/recall/F1, the confusion matrix and the classification report; extract false positives and false negatives.
9. Save figures, tables, the cleaned dataset and the trained vectorizer/model; score demo and user-supplied messages.

4.6 Safeguards: leakage and reproducibility

- Duplicates removed before splitting - identical texts cannot straddle train and test.
- Vocabulary fitted on training text only - the test set is genuinely unseen.
- Fixed seed, fixed cleaning policy, recorded library versions (Appendix B) - the run is repeatable digit for digit.
- If the seed, data version or any parameter changes, the new outputs must be recorded as a new run, not edited over the old ones.

Chapter 5 - Python Implementation

5.1 Environment and libraries

The project is delivered twice in equivalent forms: a script (sms_spam_detection.py) for one-command execution, and a Jupyter/Colab notebook (SMS_Spam_Detection_KavinRajSK_25BCE2866.ipynb) split into narrated steps for demonstration. Both run the identical pipeline with the identical seed, and their outputs agree. If a package is missing: %pip install -q ucimlrepo pandas numpy matplotlib scikit-learn

Table 8. Libraries and their jobs (as named in the activity sheet).

Library	Purpose in this project
pandas	Load, clean and summarise the dataset; all count-based probability tables
numpy	Expectation, variance and standard deviation of Y; numeric checks
matplotlib	Class-distribution, word-probability, length and confusion-matrix figures
scikit-learn	train_test_split, CountVectorizer, MultinomialNB and the evaluation metrics
ucimlrepo	Primary route for downloading dataset ID 228 from UCI (ZIP fallback included)
re	Case-insensitive whole-word matching for the probability analysis
joblib	Persisting the trained vectorizer and model as deliverable artefacts

5.2 How the program is organised

Table 9. Main components of sms_spam_detection.py.

Component	Responsibility
load_sms_dataset()	Fetch via ucimlrepo (ID 228); fall back to the bundled UCI ZIP.
clean_dataset()	Missing-value-safe normalisation, de-duplication and label encoding.
contains_whole_word()	Regex whole-word, case-insensitive matcher used for every word count.
word_probability_table()	P(Word), class-conditional and posterior probabilities plus lift.
save_*_chart() functions	Write the four report figures to figures/ with a shared style.
train_and_evaluate()	Stratified split, vectorization, training, metrics and error extraction.
predict_sms()	Score one new message; print label and both class probabilities.
interactive_test()	Optional prompt loop (python sms_spam_detection.py --interactive).

5.3 Running it

1. Script: python sms_spam_detection.py - runs the full analysis, saves every artefact and prints the five demo predictions. Add --interactive to type your own messages afterwards.
2. Notebook: open SMS_Spam_Detection_KavinRajSK_25BCE2866.ipynb in Jupyter or Google Colab and run top to bottom. In Colab either keep the data/ folder beside it or let the loader download the UCI ZIP.
3. Outputs land in data/ (cleaned dataset), figures/ (PNG charts), model/ (fitted vectorizer + classifier) and outputs/ (CSV tables, results.json and a console transcript).

5.4 Inputs and outputs

Input: the labelled UCI corpus (label, message). Statistical outputs: priors, five-word and bonus-word probability tables, the Bayes verification, E(Y), Var(Y) and length statistics. Model outputs: the fitted CountVectorizer and MultinomialNB objects, test metrics, the confusion matrix, error lists, and - for any supplied string - a predicted label with P(Spam | Message) and P(Ham | Message) that sum to 1. The complete listing is Appendix A.

Chapter 6 - Model Evaluation and Results

6.1 Training and test partitions

Table 10. Stratified split actually used (seed 2866).

Partition	Total	Ham (0)	Spam (1)	Share
Training	4,128	3,614	514	80%
Testing	1,032	904	128	20%
Cleaned total	5,160	4,518	642	100%

The vocabulary learned from the training partition contains 7,597 tokens. Neither the vocabulary nor the classifier parameters saw any test message during fitting.

6.2 Metrics, with the arithmetic shown

With TP = spam caught, TN = ham correctly passed, FP = ham wrongly flagged and FN = spam wrongly passed, the four headline measures are accuracy = (TP+TN)/all, precision = TP/(TP+FP), recall = TP/(TP+FN) and F1 = the harmonic mean of precision and recall [6]. Substituting the test counts (TP, FP, FN, TN) from Section 6.3:

Table 11. Headline metrics worked from the confusion-matrix counts.

Metric	Calculation on this test set	Result
Accuracy	(118 + 902) / 1,032	0.988372 = 98.84%
Spam precision	118 / (118 + 2) = 118 / 120	0.983333 = 98.33%
Spam recall	118 / (118 + 10) = 118 / 128	0.921875 = 92.19%
Spam F1-score	Harmonic mean of precision and recall	0.951613 = 95.16%

In words: when this model says 'spam', it is right 98 times out of 100; of every 100 real spam messages it catches about 92. Both errors have a human cost - a false positive can hide a genuine message, a false negative lets the junk through - which is why the pair is reported together instead of hiding behind the flattering accuracy figure on an 87.6%-ham test set.

6.3 Confusion matrix

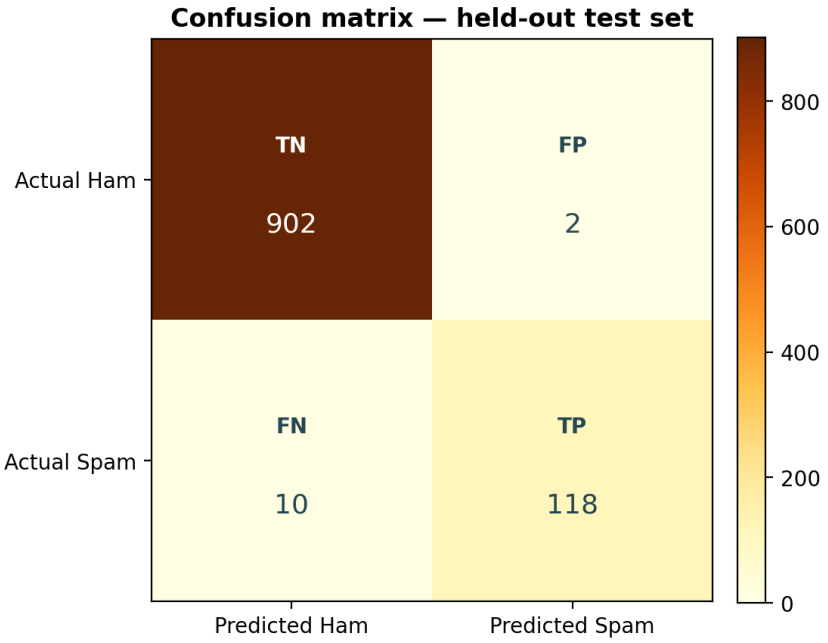


Figure 5. Confusion matrix on the held-out test set (rows = actual, columns = predicted). Only 12 of 1,032 predictions are wrong.

Table 12. Confusion matrix in words.

Actual class	Predicted Ham	Predicted Spam	Interpretation
Ham (0)	902 (TN)	2 (FP)	902 genuine messages passed correctly; 2 genuine messages were wrongly flagged as spam.
Spam (1)	10 (FN)	118 (TP)	118 spam messages were caught; 10 slipped through as if genuine.

In scikit-learn's [0, 1] label order the matrix is [[902, 2], [10, 118]], totalling 902 + 2 + 10 + 118 = 1,032 test messages. The asymmetry is the story: the model is extremely reluctant to accuse a genuine message (just 2 false positives, ham recall 99.78%), and pays for that caution by missing 10 disguised spam messages.

6.4 Classification report

Table 13. Per-class performance (scikit-learn classification_report).

Class	Precision	Recall	F1-score	Support
Ham (0)	0.9890	0.9978	0.9934	904
Spam (1)	0.9833	0.9219	0.9516	128
Macro average	0.9862	0.9598	0.9725	1,032
Weighted average	0.9883	0.9884	0.9882	1,032

The weighted average (0.988) is dominated by the 904 ham messages, as weighted averages on imbalanced data always are. The spam row is the honest summary of the harder job, and the macro average - treating both classes as equals - sits between the two.

6.5 Error analysis: reading the 12 mistakes

The test set produced 10 false negatives and only 2 false positives. Every false positive and a sample of false negatives are saved to outputs/ with their predicted probabilities; Table 14 discusses representative cases (lightly trimmed for space).

Table 14. What the errors look like, and why they happen.

Error	Message (excerpt)	Model's P(Spam)	Likely reason
False negative	"Not heard from U4 a while. Call me now am here all night..."	~0.0000	Chat-style wording and text-speak; none of the usual promotional tokens are present.
False negative	"Ever thought about living a good life with a perfect partner? Just txt back NAME and AGE..."	0.1290	Question-led dating-service spam written like friendly advice.
False negative	"LookAtMe!: Thanks for your purchase of a video clip... you've been charged 35p..."	0.0009	Reads as a transaction receipt; service vocabulary dominates.
False negative	"Your next amazing xxx PICSFREE1 video will be sent to you enjoy!..."	0.0101	Obfuscated tokens (PICSFREE1) are not whole words the model knows well.
False negative	"RCT' THNQ Adrian for U text. Rgds Vatian"	0.3851	Extremely short, garbled text - almost no evidence either way; the closest miss.
False positive	"We have sent JD for Customer Service cum Accounts Executive to ur mail id, For details contact us"	0.8667	Formal job/recruitment phrasing overlaps with commercial spam patterns.
False positive	"Waiting for your call."	0.5885	Three common words, one of which (call) leans spam in this corpus; too little context to rescue it.

The pattern is coherent: the model fails exactly where word counts fail - spam that imitates conversation or receipts, obfuscated spellings, very short messages, and genuine but formal service texts. A bag of words sees neither sender, links, timing nor word order, and a high predicted probability remains an estimate from this corpus, not proof that a message is malicious.

Chapter 7 - Real-Time SMS Testing

7.1 The message specified in the activity sheet

The sheet's example - "Congratulations! You have won a free prize. Claim now." - was passed through the fitted vectorizer and model exactly as a live message would be:

Table 15. Model output for the specified test message.

Output	Result from my fitted model
Prediction	Spam
P(Spam | Message)	0.999999641 (about 99.999964%)
P(Ham | Message)	0.000000359 (about 0.000036%)

The activity sheet illustrates the display format with 0.96 and 0.04. Those are example values, not measurements. The near-1 probability above is what my model computes, and it is near 1 for an unsurprising reason: won, free, prize and claim each carry likelihoods heavily favouring spam, and Naive Bayes multiplies them. Such extreme posteriors are typical of Naive Bayes and should be read as 'overwhelmingly spam-like', not as a calibrated 99.99996% real-world certainty (see 8.4).

7.2 Four more messages of my own design

To check the system in both directions - and on a case that fools many naive keyword filters, the genuine one-time password - I scored four additional messages:

Table 16. Additional real-time tests (full texts in outputs/realtime_test_results.csv).

Test message (intent)	Prediction	P(Spam | Message)	P(Ham | Message)
Account-block scare with urgent/offer/claim	Spam	0.999995942	0.000004058
Ordinary lunch plan	Ham (genuine)	0.000001574	0.999998426
Genuine OTP (security message that must NOT be blocked)	Ham (genuine)	0.000203184	0.999796816
Holiday-prize promotion	Spam	0.999999998	0.000000002

- "URGENT! Your account will be blocked. Verify your details now to claim your offer." - Spam, P = 0.999995942. The urgency-and-threat wording is decisive.
- "Hi, are we still meeting for lunch tomorrow at 1?" - Ham, P(Spam) = 0.000001574. Everyday words in ordinary proportions.
- "Your OTP is 482913. Please do not share it with anyone." - Ham, P(Spam) = 0.000203184. Reassuring: the most consequential genuine messages survive, though a production filter should still treat OTP senders with special care rather than trusting a word model.
- "WINNER! You have been selected for a free holiday. Call now to claim your prize!" - Spam, P = 0.999999998, the most confident of all five tests.

7.3 Testing a message yourself

1. Run the script with --interactive, or run the notebook's Step 8.
2. Type or paste any SMS text; read the predicted label and the two probabilities.
3. Use exit/quit to stop the script loop. In the notebook, edit my_message in the last code cell and re-run it.
4. Never create a fresh vectorizer for a new message - always vectorizer.transform([message]) with the fitted one.

Chapter 8 - Results and Discussion

8.1 The headline results in one place

Table 17. Summary of the main findings.

Component	Result	Meaning
Priors	P(Spam) = 0.1244; P(Ham) = 0.8756	Roughly 1 in 8 cleaned messages is spam.
Strongest word	P(Spam | Prize) = 1.0000 (70/70)	Every prize-message in this corpus is spam.
Worked example	P(Spam | Free) = 0.7236181 by count and by Bayes	The theorem reproduces the counting exactly.
Expectation / variance	E(Y) = 0.1244; Var(Y) = 0.1089	Bernoulli behaviour of the spam indicator.
Test performance	98.84% accuracy; spam P/R/F1 = 98.33% / 92.19% / 95.16%	1020 of 1,032 unseen messages classified correctly.
Errors	2 false positives, 10 false negatives	The model protects genuine mail first.
Specified SMS	Spam; P(Spam) = 0.999999641	A textbook accumulation of spam-likelihood words.

8.2 How the statistics and the classifier fit together

Chapter 3 and Chapter 6 are the same idea at two scales. The prior says how the classes balance before evidence. A one-word conditional probability shows how a single piece of evidence moves that balance - free moves the spam share from 12.44% to 72.36%, a lift of x5.82. Bayes' theorem proves the move can be computed from either direction. Expectation and variance then summarise the label itself as a random quantity. Multinomial Naive Bayes simply refuses to stop at one word: it multiplies a likelihood for every token in the message by the same kind of prior, normalises, and reports the result as P(Spam | Message). That is why the probabilities of Chapter 7 are so much more extreme than any row of Table 4 - they are products of many pieces of evidence, not one - and why the word tables remain valuable: they make the machinery inspectable at human scale.

8.3 Strengths of the approach

- Transparent: every probability can be traced to an explicit count with a stated denominator.
- Reproducible: recorded cleaning, seed and versions; one command rebuilds every number and figure.
- Cheap and fast: a sparse count matrix and a closed-form Naive Bayes fit run in seconds.
- Honestly evaluated: held-out data, per-class metrics, a confusion matrix and read-out-loud error examples.
- Immediately usable: the saved vectorizer/model pair scores any new message with probabilities, not just a label.

8.4 Limitations and validity considerations

- Historical English corpus: modern spam uses fresh vocabulary, links, other languages and deliberate obfuscation (the PICSFREE1 miss is a small preview).
- Corpus priors and word probabilities travel with the dataset; another population needs its own counts.
- Independence is false: words are not independent given the class, so posteriors are over-confident and should be calibrated before any risk-based decision uses them.
- One split: a different seed or an external test set would move the metrics somewhat; only stratified cross-validation would quantify that spread.
- Short and ambiguous texts are intrinsically hard ('Waiting for your call.'); more context - sender reputation, links, conversation history - is what production systems add, with corresponding privacy care.

Chapter 9 - Conclusion and Future Scope

9.1 Conclusion

This project built a complete probabilistic SMS spam detector on the UCI SMS Spam Collection, in the order the BAMAT207 activity demands: statistics first, classifier second. Cleaning reduced 5,574 raw messages to 5,160 unique texts (4,518 ham, 642 spam), giving priors P(Ham) = 0.8755814 and P(Spam) = 0.1244186. Among messages containing free, 72.36% are spam - a figure obtained by direct counting and reproduced exactly by Bayes' theorem - while prize, urgent, win and offer point still more strongly to spam. The Bernoulli label Y has E(Y) = 0.1244186 and Var(Y) = 0.1089386.

A Multinomial Naive Bayes classifier over 7,597 CountVectorizer tokens, trained on a stratified 80:20 split with seed 2866, classified unseen messages with 98.84% accuracy (spam precision 98.33%, recall 92.19%, F1 95.16%; confusion matrix [[902, 2], [10, 118]]). It labels the specified example as spam with P(Spam | Message) = 0.999999641, passes an ordinary OTP as ham, and its dozen mistakes are

concentrated exactly where theory predicts word counts to struggle. Probability, handled carefully and checked numerically at each step, turns out to catch a great deal of spam.

9.2 Future scope

- Stratified k-fold or repeated-split evaluation to attach variability estimates to the metrics.
- A second, recent SMS corpus for external validation against vocabulary and campaign drift.
- Character n-gram features to resist obfuscated spellings, and a comparison with logistic regression / linear SVM baselines.
- Probability calibration (e.g. on a validation split) and cost-aware thresholds that weigh a hidden OTP far above a missed advertisement.
- A small, privacy-respecting demo interface around the saved model, with versioned releases on GitHub.

9.3 What this project taught me

The activity's learning outcome promises a connection between probability theory and a working AI system, and building it made the connection concrete: a prior is a count, evidence is counts within classes, Bayes' theorem is bookkeeping that must balance to the last digit, and a classifier is that bookkeeping scaled to thousands of words. The second outcome - beginning a public digital profile - is addressed by packaging the notebook, code, figures and this report for a GitHub repository whose README explains how to reproduce every result (see Appendix C).

References

[1] Almeida, T. A. and Hidalgo, J. M. G. (2011). SMS Spam Collection [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5CC84

[2] UCI Machine Learning Repository (n.d.). SMS Spam Collection (Dataset ID 228). https://archive.ics.uci.edu/dataset/228/sms+spam+collection (accessed 27 October 2026).

[3] UCI Machine Learning Repository (n.d.). ucimlrepo: Python API for retrieving UCI datasets. https://github.com/uci-ml-repo/ucimlrepo (accessed 27 October 2026).

[4] Scikit-learn developers (n.d.). CountVectorizer - API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html (accessed 27 October 2026).

[5] Scikit-learn developers (n.d.). MultinomialNB - API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html (accessed 27 October 2026).

[6] Scikit-learn developers (n.d.). Model evaluation: scoring, metrics and classification reports. https://scikit-learn.org/stable/modules/model_evaluation.html (accessed 27 October 2026).

[7] The pandas development team (n.d.). pandas documentation. https://pandas.pydata.org/docs/ (accessed 27 October 2026).

[8] The NumPy developers (n.d.). NumPy documentation - statistics routines (mean, var). https://numpy.org/doc/stable/reference/routines.statistics.html (accessed 27 October 2026).

[9] Python Software Foundation (n.d.). re - Regular expression operations. https://docs.python.org/3/library/re.html (accessed 27 October 2026).

[10] Matplotlib developers (n.d.). Matplotlib documentation. https://matplotlib.org/stable/ (accessed 27 October 2026).

Appendix A - Complete Python Code (sms_spam_detection.py)

The listing below is the exact script submitted with this report. The notebook implements the same steps in narrated cells.

```
"""
Probabilistic SMS Spam Detection
BAMAT207 - Probability and Statistics (Project Activity)

Author          : Kavin Raj SK
Registration No.: 25BCE2866
Programme       : B.Tech, CSE - Core

What this program does
----------------------
1. Loads the UCI SMS Spam Collection (dataset ID 228) with ``ucimlrepo``
   and falls back to the local UCI ZIP file when there is no internet.
2. Cleans the corpus (missing-value check, whitespace trimming,
   duplicate removal) and encodes the labels as Spam = 1, Ham = 0.
3. Computes the empirical prior probabilities P(Spam) and P(Ham).
4. Computes word-level conditional probabilities P(Spam | Word) for the
   five suspicious words named in the activity sheet
   (free, prize, win, urgent, offer) and verifies P(Spam | Free)
   with Bayes' theorem.
5. Treats the label as a Bernoulli random variable Y and reports
   E(Y) and Var(Y).
6. Trains a Multinomial Naive Bayes classifier on CountVectorizer
   word-count features using a stratified 80:20 split.
7. Evaluates the classifier (accuracy, precision, recall, F1-score,
   confusion matrix, classification report) and inspects its mistakes.
8. Scores new SMS messages in real time and prints
   P(Spam | Message) and P(Ham | Message).

Design choice - random seed
---------------------------
RANDOM_STATE is set to 2866, the last four digits of my registration
number (25BCE2866). Any fixed seed makes the split reproducible;
using my own number simply makes this run unmistakably mine.

Run:
    python sms_spam_detection.py              # full analysis + demo tests
    python sms_spam_detection.py --interactive  # then type your own SMS
"""

from __future__ import annotations

import argparse
import io
import json
import re
import zipfile
from pathlib import Path

import joblib
import matplotlib

matplotlib.use("Agg")  # save figures even without a display (Colab-safe too)
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
)
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

# ---------------------------------------------------------------------------
# Configuration
# ---------------------------------------------------------------------------
RANDOM_STATE = 2866          # last four digits of Reg. No. 25BCE2866
TEST_SIZE = 0.20             # 80% train / 20% test, as specified
TARGET_WORDS = ["free", "prize", "win", "urgent", "offer"]   # from activity sheet
BONUS_WORDS = ["claim", "call", "text"]                      # my own extra check
BASE_DIR = Path(__file__).resolve().parent
DATA_DIR = BASE_DIR / "data"
FIGURE_DIR = BASE_DIR / "figures"
MODEL_DIR = BASE_DIR / "model"
OUTPUT_DIR = BASE_DIR / "outputs"

# Chart palette (kept identical in the written report)
COLOUR_HAM = "#2A9D8F"       # teal  -> genuine messages
COLOUR_SPAM = "#E76F51"      # burnt orange -> spam
COLOUR_DARK = "#264653"      # deep slate for titles/annotations

plt.rcParams.update({
    "font.size": 10,
    "axes.titlesize": 12,
    "axes.titleweight": "bold",
    "axes.labelsize": 10,
    "figure.dpi": 200,
})
# ---------------------------------------------------------------------------
```

```
# 1. Dataset loading
# ---------------------------------------------------------------------------
def _read_zip_member(zip_path: Path) -> pd.DataFrame:
    """Read the tab-separated SMSSpamCollection file inside the UCI ZIP."""
    with zipfile.ZipFile(zip_path) as archive:
        member = next(n for n in archive.namelist() if n.endswith("SMSSpamCollection"))
        raw_bytes = archive.read(member)
    return pd.read_csv(
        io.BytesIO(raw_bytes), sep="\t", header=None,
        names=["label", "message"], quoting=3, encoding="utf-8",
    )


def load_sms_dataset() -> tuple[pd.DataFrame, str]:
    """Try the UCI API first; fall back to a local ZIP so the work is offline-safe."""
    try:
        from ucimlrepo import fetch_ucirepo

        fetched = fetch_ucirepo(id=228)
        frame = pd.concat([fetched.data.features, fetched.data.targets], axis=1)
        # The API column names can vary, so locate the label column by its values.
        label_col = None
        for col in frame.columns:
            values = set(frame[col].astype(str).str.lower().unique())
            if values.issubset({"ham", "spam"}) and len(values) == 2:
                label_col = col
                break
        if label_col is None:
            raise ValueError("Could not identify the ham/spam column from the API response.")
        text_col = next(c for c in frame.columns if c != label_col)
        data = frame[[label_col, text_col]].copy()
        data.columns = ["label", "message"]
        return data, "UCI Machine Learning Repository via ucimlrepo (dataset ID 228)"
    except Exception as api_error:  # noqa: BLE001 - any failure should trigger the fallback
        print(f"[info] Online UCI fetch unavailable ({api_error}). Using local ZIP instead.")

    candidates = [
        DATA_DIR / "sms+spam+collection.zip",
        BASE_DIR / "sms+spam+collection.zip",
        Path.cwd() / "sms+spam+collection.zip",
        Path.cwd() / "data" / "sms+spam+collection.zip",
        Path("/content/sms+spam+collection.zip"),  # Google Colab working folder
    ]
    for candidate in candidates:
        if candidate.exists():
            return _read_zip_member(candidate), f"Local UCI ZIP file: {candidate}"
    raise FileNotFoundError(
        "Dataset not found. Install ucimlrepo with internet access, or place "
        "'sms+spam+collection.zip' in the data/ folder next to this script."
    )


# ---------------------------------------------------------------------------
# 2. Cleaning
# ---------------------------------------------------------------------------
def clean_dataset(raw: pd.DataFrame) -> pd.DataFrame:
    """Normalise labels/text, drop unusable rows and repeated messages, encode labels."""
    data = raw.copy()
    data["label"] = data["label"].astype(str).str.strip().str.lower()
    data["message"] = data["message"].astype(str).str.strip()
    data = data[data["label"].isin(["ham", "spam"])]
    data = data[data["message"] != ""]
    data = data.drop_duplicates(subset=["message"], keep="first")
    data = data.reset_index(drop=True)
    data["label_encoded"] = data["label"].map({"ham": 0, "spam": 1}).astype(int)
    return data


# ---------------------------------------------------------------------------
# 3. Word-level probability analysis
# ---------------------------------------------------------------------------
def contains_whole_word(series: pd.Series, word: str) -> pd.Series:
    """True where the complete word occurs at least once (case-insensitive)."""
    pattern = rf"\b{re.escape(word)}\b"
    return series.str.contains(pattern, case=False, regex=True)


def word_probability_table(data: pd.DataFrame, words: list[str]) -> pd.DataFrame:
    """Count-based conditional probabilities for each suspicious word."""
    total = len(data)
    n_spam = int((data["label"] == "spam").sum())
    n_ham = int((data["label"] == "ham").sum())
    p_spam = n_spam / total
    rows = []
    for word in words:
        has_word = contains_whole_word(data["message"], word)
        with_word = int(has_word.sum())
        spam_with = int((has_word & (data["label"] == "spam")).sum())
        ham_with = int(with_word - spam_with)
        p_spam_given_word = spam_with / with_word if with_word else float("nan")
        rows.append({
            "Word": word,
            "Messages containing word": with_word,
            "Spam messages": spam_with,
            "Ham messages": ham_with,
            "P(Word)": with_word / total,
            "P(Word | Spam)": spam_with / n_spam,
            "P(Word | Ham)": ham_with / n_ham,
            "P(Spam | Word)": p_spam_given_word,
```

```
            "P(Ham | Word)": ham_with / with_word if with_word else float("nan"),
            # Lift: how many times more likely spam becomes once the word is seen.
            "Lift over prior": p_spam_given_word / p_spam if with_word else float("nan"),
        })
    return pd.DataFrame(rows)


# ---------------------------------------------------------------------------
# 4. Figures
# ---------------------------------------------------------------------------
def save_class_distribution(data: pd.DataFrame) -> Path:
    counts = data["label"].value_counts().reindex(["ham", "spam"])
    total = len(data)
    fig, ax = plt.subplots(figsize=(6.4, 4.4))
    bars = ax.bar(
        ["Ham\n(genuine)", "Spam\n(unwanted)"],
        [counts["ham"], counts["spam"]],
        color=[COLOUR_HAM, COLOUR_SPAM], edgecolor="white", width=0.55,
    )
    for bar, label in zip(bars, ["ham", "spam"]):
        value = int(counts[label])
        ax.text(bar.get_x() + bar.get_width() / 2, value + total * 0.012,
                f"{value:,}\n({value / total:.2%})", ha="center", va="bottom",
                fontsize=10, color=COLOUR_DARK, fontweight="bold")
    ax.set_ylim(0, counts.max() * 1.22)
    ax.set_ylabel("Number of messages")
    ax.set_title(f"Class distribution after cleaning (N = {total:,})")
    ax.spines[["top", "right"]].set_visible(False)
    ax.yaxis.grid(alpha=0.25)
    ax.set_axisbelow(True)
    fig.tight_layout()
    path = FIGURE_DIR / "class_distribution.png"
    fig.savefig(path, bbox_inches="tight")
    plt.close(fig)
    return path


def save_word_chart(table: pd.DataFrame) -> Path:
    ordered = table.sort_values("P(Spam | Word)")
    fig, ax = plt.subplots(figsize=(6.8, 4.2))
    bars = ax.barh(ordered["Word"], ordered["P(Spam | Word)"],
                   color=COLOUR_SPAM, edgecolor="white", height=0.58)
    for bar, (_, row) in zip(bars, ordered.iterrows()):
        ax.text(min(bar.get_width() + 0.02, 1.02), bar.get_y() + bar.get_height() / 2,
                f"{row['P(Spam | Word)']:.2%}  ({int(row['Spam messages'])}/"
                f"{int(row['Messages containing word'])})",
                va="center", fontsize=9, color=COLOUR_DARK)
    ax.set_xlim(0, 1.28)
    ax.set_xlabel("P(Spam | Word)  —  share of messages containing the word that are spam")
    ax.set_title("How strongly does each suspicious word point to spam?")
    ax.spines[["top", "right"]].set_visible(False)
    ax.xaxis.grid(alpha=0.25)
    ax.set_axisbelow(True)
    fig.tight_layout()
    path = FIGURE_DIR / "word_spam_probability.png"
    fig.savefig(path, bbox_inches="tight")
    plt.close(fig)
    return path


def save_length_chart(data: pd.DataFrame) -> Path:
    lengths = data.copy()
    lengths["chars"] = lengths["message"].str.len()
    fig, ax = plt.subplots(figsize=(6.8, 4.2))
    bins = np.arange(0, 325, 15)
    for label, colour, name in [("ham", COLOUR_HAM, "Ham"), ("spam", COLOUR_SPAM, "Spam")]:
        subset = lengths.loc[lengths["label"] == label, "chars"].clip(upper=320)
        ax.hist(subset, bins=bins, alpha=0.62, color=colour, label=name, edgecolor="white")
        ax.axvline(subset.median(), color=colour, linestyle="--", linewidth=1.6)
    ax.set_xlabel("Message length (characters; values above 320 grouped at the edge)")
    ax.set_ylabel("Number of messages")
    ax.set_title("Message length by class (dashed lines = class medians)")
    ax.legend(frameon=False)
    ax.spines[["top", "right"]].set_visible(False)
    ax.yaxis.grid(alpha=0.25)
    ax.set_axisbelow(True)
    fig.tight_layout()
    path = FIGURE_DIR / "message_length_by_class.png"
    fig.savefig(path, bbox_inches="tight")
    plt.close(fig)
    return path
def save_confusion_matrix(cm: np.ndarray) -> Path:
    fig, ax = plt.subplots(figsize=(5.4, 4.6))
    image = ax.imshow(cm, cmap="YlOrBr", vmin=0)
    ax.set_xticks([0, 1], labels=["Predicted Ham", "Predicted Spam"])
    ax.set_yticks([0, 1], labels=["Actual Ham", "Actual Spam"])
    tags = [["TN", "FP"], ["FN", "TP"]]
    for (row, col), value in np.ndenumerate(cm):
        ax.text(col, row - 0.12, tags[row][col], ha="center", va="center",
                color="white" if value > cm.max() * 0.55 else COLOUR_DARK,
                fontsize=10, fontweight="bold")
        ax.text(col, row + 0.16, f"{int(value):,}", ha="center", va="center",
                color="white" if value > cm.max() * 0.55 else COLOUR_DARK, fontsize=13)
    ax.set_title("Confusion matrix — held-out test set")
    fig.colorbar(image, ax=ax, fraction=0.046, pad=0.04)
    fig.tight_layout()
    path = FIGURE_DIR / "confusion_matrix.png"
    fig.savefig(path, bbox_inches="tight")
    plt.close(fig)
```

```
    return path


# ---------------------------------------------------------------------------
# 5. Model training and evaluation
# ---------------------------------------------------------------------------
def train_and_evaluate(data: pd.DataFrame) -> dict:
    X_text = data["message"]
    y = data["label_encoded"]
    X_train_text, X_test_text, y_train, y_test = train_test_split(
        X_text, y, test_size=TEST_SIZE, random_state=RANDOM_STATE, stratify=y,
    )
    # Fit the vocabulary on training messages ONLY (prevents test-set leakage).
    vectorizer = CountVectorizer(lowercase=True)
    X_train = vectorizer.fit_transform(X_train_text)
    X_test = vectorizer.transform(X_test_text)

    model = MultinomialNB()
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    probabilities = model.predict_proba(X_test)
    ham_col = list(model.classes_).index(0)
    spam_col = list(model.classes_).index(1)

    metrics = {
        "Accuracy": float(accuracy_score(y_test, y_pred)),
        "Spam precision": float(precision_score(y_test, y_pred, pos_label=1, zero_division=0)),
        "Spam recall": float(recall_score(y_test, y_pred, pos_label=1, zero_division=0)),
        "Spam F1-score": float(f1_score(y_test, y_pred, pos_label=1, zero_division=0)),
        "Ham precision": float(precision_score(y_test, y_pred, pos_label=0, zero_division=0)),
        "Ham recall": float(recall_score(y_test, y_pred, pos_label=0, zero_division=0)),
        "Ham F1-score": float(f1_score(y_test, y_pred, pos_label=0, zero_division=0)),
    }
    cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
    report_text = classification_report(
        y_test, y_pred, labels=[0, 1],
        target_names=["Ham (0)", "Spam (1)"], digits=4, zero_division=0,
    )
    report_dict = classification_report(
        y_test, y_pred, labels=[0, 1],
        target_names=["Ham (0)", "Spam (1)"], digits=6, zero_division=0, output_dict=True,
    )
    errors = pd.DataFrame({
        "message": X_test_text.to_numpy(),
        "actual": y_test.to_numpy(),
        "predicted": y_pred,
        "P(Ham)": probabilities[:, ham_col],
        "P(Spam)": probabilities[:, spam_col],
    }, index=X_test_text.index)

    return {
        "vectorizer": vectorizer,
        "model": model,
        "X_train_text": X_train_text, "X_test_text": X_test_text,
        "y_train": y_train, "y_test": y_test, "y_pred": y_pred,
        "metrics": metrics, "confusion_matrix": cm,
        "classification_report": report_text, "report_dict": report_dict,
        "errors": errors,
        "false_negatives": errors[(errors["actual"] == 1) & (errors["predicted"] == 0)],
        "false_positives": errors[(errors["actual"] == 0) & (errors["predicted"] == 1)],
        "vocabulary_size": len(vectorizer.get_feature_names_out()),
    }


# ---------------------------------------------------------------------------
# 6. Real-time prediction
# ---------------------------------------------------------------------------
def predict_sms(message: str, vectorizer: CountVectorizer, model: MultinomialNB) -> dict:
    """Score one new SMS with the already-fitted pipeline and print both probabilities."""
    features = vectorizer.transform([message])
    probs = model.predict_proba(features)[0]
    by_class = {int(c): float(p) for c, p in zip(model.classes_, probs)}
    predicted = int(model.predict(features)[0])
    label = "Spam" if predicted == 1 else "Ham (genuine)"
    print(f"\nSMS: {message}")
    print(f"Prediction: {label}")
    print(f"P(Spam | Message) = {by_class[1]:.9f}")
    print(f"P(Ham | Message)  = {by_class[0]:.9f}")
    return {"message": message, "prediction": label,
            "P(Spam | Message)": by_class[1], "P(Ham | Message)": by_class[0]}


def interactive_test(vectorizer: CountVectorizer, model: MultinomialNB) -> None:
    print("\nType an SMS to test it. Enter 'exit' or 'quit' to stop.")
    while True:
        try:
            message = input("Enter an SMS message: ")
        except EOFError:
            break
        if message.strip().lower() in {"exit", "quit"}:
            break
        if not message.strip():
            print("Please enter a non-empty message.")
            continue
        predict_sms(message, vectorizer, model)


# ---------------------------------------------------------------------------
# Main pipeline
```

```
# ---------------------------------------------------------------------------
def main() -> None:
    parser = argparse.ArgumentParser(description="Probabilistic SMS spam detection")
    parser.add_argument("--interactive", action="store_true",
                        help="after the demo messages, prompt for your own SMS text")
    args = parser.parse_args()

    for folder in (DATA_DIR, FIGURE_DIR, MODEL_DIR, OUTPUT_DIR):
        folder.mkdir(parents=True, exist_ok=True)

    # --- Activity 1: dataset handling ---------------------------------
    raw, source = load_sms_dataset()
    print("=" * 72)
    print("PROBABILISTIC SMS SPAM DETECTION - Kavin Raj SK (25BCE2866)")
    print("=" * 72)
    print(f"Dataset source : {source}")
    print(f"Raw shape      : {raw.shape}")
    print("Missing values in the raw data:")
    print(raw.isna().sum().to_string())
    raw_counts = raw["label"].astype(str).str.strip().str.lower().value_counts()

    data = clean_dataset(raw)
    counts = data["label"].value_counts().reindex(["ham", "spam"], fill_value=0)
    total = len(data)
    duplicates_removed = len(raw) - total
    print(f"\nCleaned shape  : {data.shape}  (duplicates removed: {duplicates_removed})")
    print(f"Duplicates left: {int(data.duplicated(subset=['message']).sum())}")
    print(counts.to_string())

    # --- Activity 2: prior probabilities ------------------------------
    p_ham = counts["ham"] / total
    p_spam = counts["spam"] / total
    print(f"\nP(Ham)  = {counts['ham']}/{total} = {p_ham:.7f} ({p_ham:.2%})")
    print(f"P(Spam) = {counts['spam']}/{total} = {p_spam:.7f} ({p_spam:.2%})")
    save_class_distribution(data)

    # Extra descriptive statistics: message length
    data = data.assign(
        char_length=data["message"].str.len(),
        word_count=data["message"].str.split().str.len(),
    )
    length_summary = (
        data.groupby("label")[["char_length", "word_count"]]
        .agg(["mean", "median", "min", "max"]).round(2)
    )
    print("\nMessage-length summary by class:")
    print(length_summary.to_string())
    save_length_chart(data)

    # --- Activity 3: word-based probabilities + Bayes -----------------
    word_table = word_probability_table(data, TARGET_WORDS)
    print("\nWord-based probability table (five suspicious words):")
    print(word_table.to_string(index=False, float_format=lambda v: f"{v:.6f}"))
    save_word_chart(word_table)

    bonus_table = word_probability_table(data, BONUS_WORDS)
    print("\nBonus words (my own extension - for contrast):")
    print(bonus_table.to_string(index=False, float_format=lambda v: f"{v:.6f}"))

    free = word_table.loc[word_table["Word"] == "free"].iloc[0]
    bayes_result = (free["P(Word | Spam)"] * p_spam) / free["P(Word)"]
    direct_result = free["Spam messages"] / free["Messages containing word"]
    print("\nBayes' theorem verification for the word 'free':")
    print(f"  P(Free | Spam) = {free['P(Word | Spam)']:.7f}")
    print(f"  P(Spam)        = {p_spam:.7f}")
    print(f"  P(Free)        = {free['P(Word)']:.7f}")
    print(f"  Bayes  P(Spam | Free) = {bayes_result:.7f}")
    print(f"  Direct count check    = {int(free['Spam messages'])}/"
          f"{int(free['Messages containing word'])} = {direct_result:.7f}")
    assert abs(bayes_result - direct_result) < 1e-9, "Bayes check failed!"

    # --- Activity 4: random variable Y --------------------------------
    y_values = data["label_encoded"].to_numpy(dtype=float)
    expectation = float(np.mean(y_values))
    variance_population = float(np.var(y_values, ddof=0))
    print("\nRandom variable Y (1 = Spam, 0 = Ham):")
    print(f"  E(Y)   = P(Spam) = {expectation:.7f}")
    print(f"  Var(Y) = p(1-p)  = {expectation * (1 - expectation):.7f}")
    print(f"  NumPy population variance check = {variance_population:.7f}")
    print(f"  SD(Y)  = {np.sqrt(variance_population):.7f}")

    # --- Activities 5-7: split, train, evaluate -----------------------
    fitted = train_and_evaluate(data)
    cm = fitted["confusion_matrix"]
    tn, fp, fn, tp = int(cm[0, 0]), int(cm[0, 1]), int(cm[1, 0]), int(cm[1, 1])
    print(f"\nStratified split (random_state={RANDOM_STATE}): "
          f"train = {len(fitted['X_train_text'])}, test = {len(fitted['X_test_text'])}")
    print(f"Vocabulary size (training text only): {fitted['vocabulary_size']:,}")
    print("\nModel performance on the held-out test set:")
    for name, value in fitted["metrics"].items():
        print(f"  {name:<16} {value:.6f}  ({value:.2%})")
    print(f"\nConfusion matrix [[TN, FP], [FN, TP]] = [[{tn}, {fp}], [{fn}, {tp}]]")
    print("\nClassification report:")
    print(fitted["classification_report"])
    save_confusion_matrix(cm)

    print(f"False negatives - spam let through as ham: {fn}")
    print(fitted["false_negatives"][["message", "P(Ham)", "P(Spam)"]].head(5).to_string(index=False))
    print(f"\nFalse positives - genuine messages flagged as spam: {fp}")
```

```
    print(fitted["false_positives"][["message", "P(Ham)", "P(Spam)"]].head(5).to_string(index=False))
    # --- Activity 8: real-time testing ---------------------------------
    demo_messages = [
        "Congratulations! You have won a free prize. Claim now.",   # activity-sheet example
        "URGENT! Your account will be blocked. Verify your details now to claim your offer.",
        "Hi, are we still meeting for lunch tomorrow at 1?",
        "Your OTP is 482913. Please do not share it with anyone.",
        "WINNER! You have been selected for a free holiday. Call now to claim your prize!",
    ]
    print("\n" + "-" * 72)
    print("REAL-TIME SMS TESTS")
    print("-" * 72)
    demo_results = [predict_sms(m, fitted["vectorizer"], fitted["model"]) for m in demo_messages]

    # --- Save every deliverable artefact -------------------------------
    data[["label", "message", "label_encoded"]].to_csv(DATA_DIR / "cleaned_dataset.csv", index=False)
    word_table.to_csv(OUTPUT_DIR / "word_probabilities.csv", index=False)
    bonus_table.to_csv(OUTPUT_DIR / "bonus_word_probabilities.csv", index=False)
    pd.DataFrame(demo_results).to_csv(OUTPUT_DIR / "realtime_test_results.csv", index=False)
    fitted["false_negatives"].to_csv(OUTPUT_DIR / "false_negatives.csv")
    fitted["false_positives"].to_csv(OUTPUT_DIR / "false_positives.csv")
    joblib.dump(fitted["vectorizer"], MODEL_DIR / "count_vectorizer.joblib")
    joblib.dump(fitted["model"], MODEL_DIR / "multinomial_nb_model.joblib")

    length_flat = {}
    for label in ["ham", "spam"]:
        subset = data[data["label"] == label]
        length_flat[label] = {
            "char_mean": float(subset["char_length"].mean()),
            "char_median": float(subset["char_length"].median()),
            "char_min": int(subset["char_length"].min()),
            "char_max": int(subset["char_length"].max()),
            "word_mean": float(subset["word_count"].mean()),
            "word_median": float(subset["word_count"].median()),
        }

    results = {
        "author": "Kavin Raj SK",
        "registration_number": "25BCE2866",
        "dataset_source": source,
        "random_state": RANDOM_STATE,
        "raw_rows": int(len(raw)),
        "raw_ham": int(raw_counts.get("ham", 0)),
        "raw_spam": int(raw_counts.get("spam", 0)),
        "raw_missing_label": int(raw["label"].isna().sum()),
        "raw_missing_message": int(raw["message"].isna().sum()),
        "duplicates_removed": int(duplicates_removed),
        "cleaned_rows": int(total),
        "cleaned_ham": int(counts["ham"]),
        "cleaned_spam": int(counts["spam"]),
        "p_ham": float(p_ham),
        "p_spam": float(p_spam),
        "expectation_Y": expectation,
        "variance_Y": variance_population,
        "sd_Y": float(np.sqrt(variance_population)),
        "word_table": word_table.to_dict(orient="records"),
        "bonus_word_table": bonus_table.to_dict(orient="records"),
        "bayes_free": {
            "p_free_given_spam": float(free["P(Word | Spam)"]),
            "p_spam": float(p_spam),
            "p_free": float(free["P(Word)"]),
            "p_free_and_spam": float(free["Spam messages"] / total),
            "p_spam_given_free_bayes": float(bayes_result),
            "p_spam_given_free_direct": float(direct_result),
        },
        "length_summary": length_flat,
        "train_size": int(len(fitted["X_train_text"])),
        "test_size": int(len(fitted["X_test_text"])),
        "train_ham": int((fitted["y_train"] == 0).sum()),
        "train_spam": int((fitted["y_train"] == 1).sum()),
        "test_ham": int((fitted["y_test"] == 0).sum()),
        "test_spam": int((fitted["y_test"] == 1).sum()),
        "vocabulary_size": int(fitted["vocabulary_size"]),
        "metrics": fitted["metrics"],
        "classification_report": fitted["report_dict"],
        "confusion_matrix": {"TN": tn, "FP": fp, "FN": fn, "TP": tp},
        "false_negative_examples": fitted["false_negatives"].head(6).to_dict(orient="records"),
        "false_positive_examples": fitted["false_positives"].head(6).to_dict(orient="records"),
        "demo_results": demo_results,
    }
    (OUTPUT_DIR / "results.json").write_text(json.dumps(results, indent=2), encoding="utf-8")
    print(f"\nArtefacts saved under: {BASE_DIR}")
    print("  data/cleaned_dataset.csv | figures/*.png | model/*.joblib | outputs/*")

    if args.interactive:
        interactive_test(fitted["vectorizer"], fitted["model"])


if __name__ == "__main__":
    main()
```

Appendix B - Verified Output Snapshot

Recorded outputs of the submitted code under the stated configuration. Re-run and compare; report any difference rather than copying these values blindly.

Table 18. Verified outputs of this run (random_state = 2866).

Output	Recorded value
Raw dataset shape	(5,574, 2) - source: local UCI ZIP fallback
Duplicates removed / cleaned rows	414 / 5,160
Cleaned ham / spam	4,518 / 642
P(Ham) / P(Spam)	0.8755814 / 0.1244186
P(Spam | Free) - direct and Bayes	0.7236181 (both methods)
E(Y) / Var(Y) / SD(Y)	0.1244186 / 0.1089386 / 0.3300585
Train / test sizes	4,128 / 1,032 (stratified)
Vocabulary size	7,597 tokens
Accuracy / spam precision / recall / F1	98.84% / 98.33% / 92.19% / 95.16%
Confusion matrix [[TN, FP], [FN, TP]]	[[902, 2], [10, 118]]
Specified SMS prediction	Spam; P(Spam) = 0.999999641; P(Ham) = 0.000000359

Environment used for the recorded run: Python 3.13.16, pandas 2.2.3, NumPy 2.3.5, Matplotlib 3.10.9, scikit-learn 1.6.1. The ucimlrepo import route for dataset 228 was unavailable in this environment, so the code automatically used the official UCI ZIP in data/; the direct-fetch code path remains in the listing for online use (e.g. Google Colab).

Appendix C - Deliverables and Submission Checklist

Table 19. Mapping the activity sheet's expected deliverables to the submitted files.

Expected deliverable	Submitted as
1. Python / Google Colab notebook	SMS_Spam_Detection_KavinRajSK_25BCE2866.ipynb (executed, with outputs) + sms_spam_detection.py
2. Cleaned dataset summary	Table 2 (Chapter 2); data/cleaned_dataset.csv; outputs/results.json
3. Class-distribution chart	Figure 1; figures/class_distribution.png (+ length and word charts)
4. Probabilities for five suspicious words	Table 4 + Figure 3 (Chapter 3); outputs/word_probabilities.csv; Bayes check in Table 5
5. Trained Naive Bayes model	model/multinomial_nb_model.joblib with model/count_vectorizer.joblib
6. Confusion matrix and performance report	Figure 5 + Tables 11-14 (Chapter 6); outputs/false_*.csv
7. GitHub repository (link at submission)	README.md, requirements.txt and GITHUB_UPLOAD_GUIDE.md included; paste your repository URL where the faculty asks for it

- Personal details verified: Kavin Raj SK, 25BCE2866, B.TECH CSE--CORE.
- Notebook run top-to-bottom; outputs match Appendix B.
- Report (DOCX + PDF), notebook, script, figures, outputs and model are in one project folder.
- Repository created and pushed; link ready to share with the faculty.
- Report and code use the same cleaning policy and seed (2866).